1 The Mathematical Essence of Attention: A Self-Dialogue

This is not a paper — it is the record of a line of reasoning. We start from a plain question, "what exactly is QK^T?", and keep pushing: each step first raises a doubt, then works through it, occasionally taking a wrong turn and correcting it. The goal is to get the causal order of attention straight: Q, K, V are not definitions handed down from above, but **products derived** from the single question of "how do we measure the relatedness between two words?"

1.1 1. Starting Point: Why is QK^T the "Relatedness Between Words"?

Most tutorials open by dumping the formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

and then tell you "Q looks for K, computes attention weights, and re-weights V." But this is **treating the conclusion as the definition** — you have no idea why Q, K exist or why QK^T represents relatedness.

Let us ask differently: **if you had to design, from scratch, a way to measure "how related word i is to word j", how would you do it?**

The most natural starting point is the inner product. Element (i, j) of QK^T is the dot product of two row vectors:

$$(QK^T)_{ij} = Q_i \cdot K_j = \sum_d Q_{id}K_{jd} = \|Q_i\|\,\|K_j\|\cos\theta$$

The geometric meaning of the inner product is **projection / degree of alignment**: the more aligned two vectors are, the larger the inner product. So the inner product is inherently a similarity measure — this is where "relatedness" comes from mathematically. **It is not that attention comes first and relatedness follows; rather, once you want to measure relatedness, the inner product emerges naturally.**

1.2 2. Q, K Come From the Same X: The Bilinear Form Enters

The crucial point is that Q and K are both linear projections of the **same input** X:

$$Q = XW_Q, \qquad K = XW_K$$

So $Q_i = x_i W_Q$ (the projected embedding of word i) and $K_j = x_j W_K$. Substituting into the inner product:

$$(QK^T)_{ij} = (x_i W_Q)(x_j W_K)^T = x_i\,(W_Q W_K^T)\,x_j^T = x_i\,M\,x_j^T$$

where

$$M = W_Q W_K^T \in \mathbb{R}^{d_{model} \times d_{model}}$$

This step is the pivot of the whole derivation. $(QK^T)_{ij}$ is nothing but a **bilinear form** of the raw embeddings x_i, x_j through a matrix M. It measures "how related words i and j are under the metric defined by M."

A natural, correct order of understanding now emerges:

> **Relatedness = bilinear form $x_i M x_j^T$ (essence) $\rightarrow$ M factorizes as $W_Q W_K^T$ (engineering) $\rightarrow$ Q, K and QK^T emerge (conclusion).**

Q, K are not the **premise** of attention; they are the **byproduct** of the idea "measure relatedness with a learnable bilinear form." Tutorials tell the story backwards.

1.3 3. A Wrong Turn: Is This a Quadratic Form? How Can It Be Asymmetric?

Here I briefly doubted myself: $x_i M x_j^T$ looks like a quadratic form, and quadratic forms are supposed to be symmetric. But attention needs "i attending to j" to differ from "j attending to i" — where does the asymmetry come from?

After working through it: it is not a quadratic form, it is a bilinear form. And that distinction is precisely the source of asymmetry.

- **Quadratic form** $x^T A x$: the **same vector** x on both sides. It is always equivalent to a symmetric matrix, because $x^T A x = x^T \frac{A+A^T}{2} x$ — the antisymmetric part is annihilated by having the same vector on both sides. **A quadratic form cannot express asymmetry.**
- **Bilinear form** $x_i M x_j^T$: **two different vectors** $x_i \neq x_j$. The antisymmetric part of M is **not** annihilated.

Rigorously checking asymmetry — swap i, j and see if it stays equal:

$$x_j M x_i^T = (x_j M x_i^T)^T = x_i M^T x_j^T$$

(a scalar equals its own transpose, then expand the transpose). Hence

$$x_i M x_j^T = x_j M x_i^T \iff M = M^T$$

But $M = W_Q W_K^T$, where W_Q, W_K are **two independent** projections, so in general $W_Q W_K^T \neq W_K W_Q^T$, i.e. $M \neq M^T$. **Therefore $x_i M x_j^T \neq x_j M x_i^T$: asymmetry holds.**

The capacity for asymmetry comes from two stacked facts: (1) two different vectors (not the single vector of a quadratic form); (2) two different projections $W_Q \neq W_K$. If we forced $W_Q = W_K$, then $M = W_Q W_Q^T$ would be symmetric and attention would degenerate to undirected — it is precisely because Q and K use different projections that we get directed attention.

> The lesson here: my initial intuition "a quadratic form should be symmetric" was not wrong; the error was in treating this as a quadratic form. It is a bilinear form, and a bilinear form is asymmetric by nature.

1.4 4. What Does "the Metric Defined by M" Actually Mean?

The phrase "relatedness under the metric defined by M" is abstract. Ground it with concrete choices of M:

$M = I$ **(identity):**

$$x_i I x_j^T = x_i \cdot x_j = \sum_k x_{ik} x_{jk}$$

Ordinary dot product, every dimension **equally weighted**. This is the standard Euclidean metric.

$M = \text{diag}(w_1, \ldots, w_d)$ **(diagonal):**

$$x_i M x_j^T = \sum_k w_k \, x_{ik} x_{jk}$$

Now each dimension has a weight w_k. If $w_1 = 10, w_2 = 0.1$, dimension 1's match is amplified and dimension 2 is nearly ignored. M **says: when judging how alike two words are, which dimensions matter.**

M **off-diagonal (general case):**

$$x_i M x_j^T = \sum_{k,l} M_{kl} \, x_{ik} x_{jl}$$

Cross-dimension terms $M_{kl} \, x_{ik} x_{jl}$ appear — "dimension k of word i" can now relate to "dimension l of word j." For example, if dimension 3 encodes tense and dimension 7 encodes subject–verb agreement, $M_{37} \neq 0$ lets "i's tense" match "j's agreement."

So "metric" mathematically means **how you define the generalized inner product of two vectors**. M can stretch some dimensions, compress others, mix dimensions. Since $M = W_Q W_K^T$ is **learned**, training turns it into "how relatedness between words should be computed for this task." That is "relatedness under the metric defined by M."

1.5 5. Settling the Dimensions

At this point the dimensions must be pinned down — this is where whiteboard errors happen most.

Tensor	Shape
X	$L \times d_{model}$
W_Q, W_K	$d_{model} \times d_k$
W_V	$d_{model} \times d_v$
Q, K	$L \times d_k$

Tensor	Shape
V	$L \times d_v$
$M = W_Q W_K^T$	$d_{model} \times d_{model}$
QK^T	$L \times L$
output	$L \times d_v$

A few constraints:

1. The output dimensions of W_Q, W_K **must match** (both d_k), otherwise the QK^T dot product is dimensionally invalid.
2. The output dimension d_v of W_V **can be independent** — it does not enter the dot product, it is only weighted and summed.
3. QK^T is $L \times L$, **determined only by sequence length, independent of** d_k — this is exactly the origin of the $O(L^2)$ memory bottleneck for long sequences.

1.6 6. The Key Insight: M Is Low-Rank — This Is a Matrix Factorization

M is $d_{model} \times d_{model}$, but its rank is capped by the bottleneck dimension d_k:

$$\text{rank}(M) = \text{rank}(W_Q W_K^T) \leq \min(\text{rank}(W_Q), \text{rank}(W_K)) \leq d_k$$

Typically $d_k \ll d_{model}$ (e.g. $d_{model} = 512, d_k = 64$). **So M is a matrix nominally 512×512 but of rank at most 64 — it is low-rank.** This is exactly the "matrix factorization" scent.

Low-rankness has three consequences:

(1) Computationally — M is never explicitly formed. By associativity:

$$x_i M x_j^T = x_i (W_Q W_K^T) x_j^T = (x_i W_Q)(x_j W_K)^T = Q_i \cdot K_j$$

Relatedness is a dot product in d_k-dimensional space, not d_{model}. Cost drops from $O(d_{model}^2)$ to $O(d_{model} d_k)$.

(2) In terms of expressiveness — low rank is an inductive bias. Relatedness is not computed in the full space; it is projected into a d_k-dimensional subspace first. The model is forced to "judge relatedness only along the d_k most relevant directions."

(3) Multi-head is the parallel concatenation of low-rank factorizations. With h heads, each $M_i = W_Q^{(i)} W_K^{(i)T}$ has rank $\leq d_{model}/h$. The overall attention is a parallel combination of h low-rank relatedness patterns, each head capturing one kind of relation (syntax, coreference, tense...) in a **different low-rank subspace**. This is more structured than a single full-rank M.

1.7 7. The SVD View: What Are the "Principal Components" of Relatedness?

Here a key question arises: since M is low-rank and SVD-like, can we look at it **from M as a whole** (rather than splitting into two towers)?

Yes — and this view is closer to the essence. Read $x_i M x_j^T$ as "M acts on x_j, then take the inner product with x_i" :

$$x_i M x_j^T = x_i \cdot (M x_j^T)$$

Mx_j^T linearly transforms x_j into a new d_{model}-dimensional vector, then takes the standard inner product with x_i.

Now take the SVD of M: $M = U\Sigma V^T$, with U, V orthogonal and $\Sigma = \text{diag}(\sigma_1, \ldots, \sigma_{d_k}, 0, \ldots, 0)$ (low-rank, only d_k nonzero singular values). Substituting:

$$x_i M x_j^T = x_i U \Sigma V^T x_j^T = \sum_{r=1}^{d_k} \sigma_r \, (x_i \cdot u_r)(x_j \cdot v_r)$$

Reading term by term:

- u_r, v_r: the r-th pair of **principal relatedness directions**;
- $x_i \cdot u_r$: the component of x_i along u_r;
- $x_j \cdot v_r$: the component of x_j along v_r;
- σ_r: the **importance weight** of this direction pair.

So relatedness = a weighted sum over d_k "principal direction pairs." Directions with large σ_r dominate; directions with $\sigma_r = 0$ (beyond the d_k-th) do not participate at all — precisely low rank: only d_k principal directions are active.

> Here we correct an earlier statement. In the "two towers" view ($Q_i K_j$), x_i, x_j each descend to d_k dimensions and meet in low dimension, **without rising back** to d_{model}. But in the "M as a whole operator" view, Mx_j^T is indeed a d_{model}-dimensional vector. So the description "project x_i down to extract principal components, regenerate a d_{model}-dimensional vector, then inner-product with x_j" is **entirely correct under the M view** — it merely splits at a different point than the two-tower view. Same $x_i M x_j^T$, two views, two dimensional narratives, mathematically equivalent.

1.8 8. A Distinction: SVD-like, But Not SVD

Low rank evokes SVD, but with a caveat:

- **SVD** $M = U\Sigma V^T$: U, V **orthogonal**, Σ **diagonal nonnegative** — a unique, constrained factorization with ordered singular values.
- $M = W_Q W_K^T$: W_Q, W_K have **no orthogonality constraint**, freely learned, factors neither unique nor orthogonal.

Both are low-rank factorizations; SVD is the most canonical one (orthogonal + diagonal), while $W_Q W_K^T$ is an unconstrained low-rank factorization. Any matrix of rank $\leq d_k$ can be written as an SVD or as $W_Q W_K^T$, but the latter' s factors are neither unique nor orthogonal. So the precise statement is: M **is low-rank; the intuition "extract d_k principal components" is right, but the factors are learned and non-orthogonal — not a strict SVD.**

1.9 9. Isomorphism With Two-Tower Retrieval

$$x_i M x_j^T = (x_i W_Q)(x_j W_K)^T = \langle Q_i, K_j \rangle$$

This structure is **exactly isomorphic** to the **two-tower** model in recommendation / retrieval:

$$\text{score } R_{ij} = \langle u_i, v_j \rangle, \quad u_i = \text{user\_tower}(x_i),\ v_j = \text{item\_tower}(x_j)$$

W_Q is the query tower, W_K the key tower, and the dot product computes relevance. In matrix factorization $R \approx UV^T$, $u_i \cdot v_j$ is a low-rank score — the same mathematical skeleton as $Q_i \cdot K_j$.

> In other words: two-tower retrieval is essentially the relatedness-computation part of single-head attention. Anyone who has built EBR / two-tower recall has, in fact, already been using the mathematical core of attention — they just never connected it explicitly to transformers.

1.10 10. The $\sqrt{d_k}$ Scaling: Why the Square Root

$$\text{scores} = \frac{QK^T}{\sqrt{d_k}}$$

Assume the entries of Q_i, K_j are independent, zero-mean, unit-variance. Then the dot product

$$Q_i \cdot K_j = \sum_{d=1}^{d_k} Q_{id} K_{jd}$$

is a sum of d_k independent terms, with **variance** $= d_k$ (variances add), so standard deviation $\sim \sqrt{d_k}$. The higher the dimension, the larger the dot product magnitude.

Without scaling, large values entering softmax make the distribution **extremely sharp** (one position near 1, the rest near 0), landing in softmax' s saturation region where the gradient ≈ 0 and training stalls. Dividing by $\sqrt{d_k}$ normalizes the variance back to 1. **The reason it is $\sqrt{d_k}$ and not d_k: what must be cancelled is the standard deviation $\sqrt{d_k}$, not the variance.**

1.11 11. Stepping Back: The Correct Order of This Derivation

Collapse the whole line into one sentence:

> **First ask "how to compute relatedness" → inner product → bilinear form** $x_i M x_j^T$ **→** M **should be low-rank (relatedness is low-dimensional) → factorize** $M = W_Q W_K^T$ **→** Q, K, QK^T **emerge → normalize (softmax +** $\sqrt{d_k}$**) → weighted aggregation of** V **→ full attention.**

Q, K, V are **products derived** from "measure relatedness with a low-rank bilinear form," not premises to memorize. Many people can recite $\text{softmax}(QK^T/\sqrt{d})V$; those who can say "attention is essentially defining word relatedness with a low-rank bilinear form, and Q, K are its low-rank factorization" understand the **why**, not just the **what**.

1.12 12. The Bigger Picture: Dimensional Transformation Is a Through-Line of ML

Finally, place attention into a larger coordinate system. Nearly every model does the same thing — **transform information into the right dimensionality so the desired structure becomes visible**. The only differences are the direction of transformation and whether it is hand-designed or learned.

SVM kernel — lift to separate. In the original space the data is not linearly separable (e.g. concentric circles). A kernel $\phi : \mathbb{R}^d \to \mathbb{R}^D$ ($D \gg d$) lifts the data to high dimension, making the tangled classes linearly separable. The kernel trick computes only the high-dimensional inner product $K(x_i, x_j) = \langle \phi(x_i), \phi(x_j) \rangle$ without materializing the lift. **Motivation: what cannot be separated in low dimension can be separated in high dimension.**

Attention — reduce to match. Embeddings live in high-dimensional d_{model}; direct matching is noisy and expensive, so we project into a d_k-dimensional subspace to compute similarity. **Motivation: high-dimensional matching is noisy and slow; reducing to principal components first makes matching sharper and faster.**

Note that SVM and attention go in **opposite directions** — one lifts for separability, the other reduces for matching — because their goals differ: SVM wants to "separate," attention wants to "measure similarity." **The direction of dimensional transformation is dictated by the task objective.**

Deep NN — alternating up and down, reshaping layer by layer. A DNN is not one-directional:

$$h_{l+1} = \sigma(W_l h_l + b_l)$$

Up-projection layers (wider hidden, e.g. the transformer FFN $d_{model} \to 4d_{model} \to d_{model}$, up then down) add capacity and manufacture separability; down-projection layers (bottlenecks, pooling) compress, denoise, abstract; the nonlinearity σ is the key — a purely linear stack of ups and downs is still linear ($W_2 W_1$ is just one matrix), and the nonlinearity makes each layer non-mergeable, enabling layer-by-layer reshaping. **Motivation: not knowing the optimal dimension, nor whether to go up or down, stack many layers, learn one transformation per layer, and let the network discover the whole "up–down–twist" sequence itself.**

A unifying view — the genealogy of kernel methods / representation learning:

- SVM kernel: a **fixed** lift ϕ (human-chosen RBF / polynomial);
- Attention: a **learned** single low-dimensional projection W_Q, W_K (a data-driven kernel);
- Deep NN: a **learned** multi-layer nonlinear transformation (ϕ itself is learned).

The evolutionary through-line is clear: **from "human-designed dimensional transformation" (SVM kernel) → "data-learned single transformation" (attention) → "data-learned multi-layer transformation" (DNN).** The further along, the less the transformation relies on human priors and the more on learning from data. This is precisely the essential progress of deep learning over classical ML — **no longer hand-designing features / kernels, but learning the transformation itself; from feature engineering to representation learning.** And in every case, the core is the same sentence: dimensional transformation makes structure visible.

Written after one round of self-dialogue.